

DOCUMENTO MAESTRO DEL TFM — v3.0

Predicción de abandono de clientes en entornos e-commerce

Septiembre de 2026 — Diseño técnico, metodológico y estado del arte

Esta versión amplía las anteriores con literatura científica reciente, criterios metodológicos y una explicación explícita de cada herramienta: para qué sirve, cómo se usará y qué riesgo del proyecto ayuda a controlar.

1. Objetivo

Construir un sistema reproducible que estime la probabilidad de abandono de clientes a partir de su comportamiento transaccional y que pueda evolucionar posteriormente a un servicio de predicción para empresas e-commerce.

La literatura reciente confirma el valor de RFM en churn e-commerce y explora su evolución temporal junto con modelos de Deep Learning. Para el TFM se recomienda empezar por modelos tabulares sólidos y solo añadir Deep Learning si aporta una mejora demostrable. ■cite■turn0search6■turn0search0■

2. Dataset: UCI Online Retail II

UCI registra 1.067.371 transacciones entre 01/12/2009 y 09/12/2011. El negocio vende principalmente artículos de regalo y UCI advierte que muchos clientes son mayoristas. Hay valores faltantes; InvoiceNo con prefijo C representa una cancelación. El dataset está bajo CC BY 4.0. ■cite■turn0search1■

Un estudio de 2026 usa este mismo dataset con 18 meses de observación y 6 meses de outcome, demostrando que es viable construir un target temporal de churn sobre UCI. ■cite■turn0search2■

Conclusión: UCI es un candidato fuerte, pero su componente mayorista limita la generalización a B2C puro. La decisión definitiva se tomará después de la auditoría real.

3. Cómo definiremos churn

Cada observación será **cliente + fecha de corte T**. Las features solo usarán información disponible hasta T. Churn=1 si no existe una compra válida durante H días después de T; churn=0 si existe al menos una. Compararemos H=60, 90 y 120 días.

La elección de H se justificará con la distribución de intervalos entre compras y, cuando sea útil, Kaplan-Meier. Así evitamos fijar 90 días arbitrariamente.

4. Snapshots y leakage

Crearemos snapshots mensuales. Un mismo cliente puede aparecer varias veces; por ello el split será temporal y estudiaremos además separación por CustomerID para evitar una estimación artificialmente optimista.

Se implementará un test automático que compruebe que ninguna transacción posterior a T entra en las features. Si falla, el modelo no podrá pasar a evaluación final.

5. Cancelaciones y devoluciones

Las operaciones con InvoiceNo que empieza por C se tratarán como cancelaciones, no como nuevas compras. Quantity negativa también se auditará. Para Monetary compararemos gasto bruto y neto; Frequency contará actividad comercial válida. La regla final se congelará tras la auditoría.

6. Feature engineering

Base RFM: Recency, Frequency y Monetary. Añadiremos número de pedidos, ticket medio, unidades, diversidad de productos, días desde primera compra, intervalos entre pedidos, tendencia de gasto y actividad reciente. Cada feature tendrá una fecha de referencia verificable.

La literatura reciente considera RFM una base central para churn y estudia también RFM variable en el tiempo.

■cite■turn0search6■turn0search0■

7. Herramientas: para qué y cómo

Python: lenguaje principal; toda la pipeline.

pandas: limpieza, joins, agregaciones y snapshots.

NumPy: cálculo numérico.

Jupyter: EDA y experimentación, pero la lógica definitiva vivirá en src.

scikit-learn: pipelines, regresión logística, Random Forest, métricas, validación y calibración.

XGBoost: candidato de gradient boosting para datos tabulares.

SHAP: explicaciones globales y locales.

Matplotlib: gráficos reproducibles.

pytest: tests de datos, modelo y anti-leakage.

Git/GitHub: historial y trazabilidad.

pyproject.toml: dependencias/versiones reproducibles.

MLflow, opcional: registro de experimentos y artefactos si el número de experimentos crece.

Docker: empaquetado reproducible.

FastAPI + Pydantic: convertir el modelo validado en API con entradas/salidas estructuradas.

PostgreSQL: persistencia en fase producto, no necesaria para el primer modelo.

Frontend: portal de cliente, únicamente después de validar el núcleo ML.

FastAPI aporta OpenAPI, validación con Pydantic y documentación interactiva Swagger/ReDoc, por lo que es adecuado para demostrar que el modelo puede consumirse como servicio. ■cite■turn1search0■turn1search6■

8. Pipeline ML

La pipeline será: datos → limpieza → snapshots → target → features → split temporal → preprocessing → entrenamiento → calibración/umbral → evaluación → SHAP → serialización → API.

Las transformaciones que aprenden parámetros deberán ajustarse solo con train. Para probabilidades de riesgo estudiaremos calibración; scikit-learn documenta CalibratedClassifierCV y métodos sigmoid/isotonic/temperature, usando datos independientes para calibrar correctamente. ■cite■turn1search3■turn1search5■

9. Desbalance

Primero mediremos la prevalencia. Después compararemos class_weight y ajuste de umbral; SMOTE/resampling, si se usa, solo dentro de train. Métricas: PR-AUC, ROC-AUC, precision, recall, F1, lift/gains y calibración. La literatura reciente muestra que el desbalance es un problema relevante en churn y estudia SMOTE y ensembles. ■cite■turn0search16■

10. Modelos

Orden: Dummy baseline → Regresión Logística → Random Forest → XGBoost/Gradient Boosting. Deep Learning será opcional. Investigaciones recientes sobre e-commerce comparan modelos tradicionales con LSTM/GRU y otras arquitecturas, pero la complejidad debe justificarse por mejora real. ■cite■turn0search5■turn0search6■

11. SHAP

SHAP responderá: ¿qué variables explican el riesgo global? y ¿por qué este cliente recibió esta predicción? Esto convierte una probabilidad en información accionable para retención. Estudios recientes de churn utilizan SHAP junto con modelos de boosting y análisis de segmentos. ■cite■turn0search14■turn0search8■

12. Supervivencia

Kaplan-Meier se utilizará como análisis temporal para estudiar cuánto tarda en aparecer el siguiente pedido y apoyar la selección de H. No sustituirá al clasificador; aporta evidencia sobre la naturaleza temporal del abandono.

13. MLOps y producto

Arquitectura objetivo: datos → validación → features → modelo → evaluación → registro/versionado → API → portal. Un trabajo de 2026 sobre MLOps de churn integra entrenamiento, despliegue, explicabilidad y monitorización de drift, respaldando este enfoque evolutivo. ■cite■turn0search18■

14. Testing obligatorio

Tests mínimos: esquema de datos; limpieza; RFM; fechas de corte; target; ausencia de datos futuros; consistencia de features; carga del modelo; predicción; validación de API; y equivalencia entre pipeline de entrenamiento y producción. El test anti-leakage será requisito de aceptación.

15. Reproducibilidad

Cada experimento registrará dataset/hash, commit Git, Python, dependencias, semilla, definición de churn, H, features, modelo, hiperparámetros y métricas. Esto permitirá reconstruir resultados para el tribunal.

16. Estructura del repositorio

```
tfm-churn-ecommerce/  
■■■ data/raw  
■■■ data/processed  
■■■ notebooks  
■■■ src/data  
■■■ src/features  
■■■ src/labels  
■■■ src/models  
■■■ src/evaluation  
■■■ src/explainability  
■■■ src/api  
■■■ tests  
■■■ models  
■■■ reports  
■■■ configs  
■■■ pyproject.toml  
■■■ Dockerfile  
■■■ README.md
```

17. Orden de ejecución

- 1 Auditar el archivo UCI real y registrar procedencia/hash.
- 2 Analizar CustomerID, cancelaciones, devoluciones, países y outliers.
- 3 Analizar intervalos entre compras y supervivencia.
- 4 Comparar 60/90/120 días y fijar H.
- 5 Diseñar snapshots y split temporal/por cliente.
- 6 Construir features y tests anti-leakage.
- 7 Entrenar baselines y modelos tabulares.
- 8 Evaluar discriminación, calibración y estabilidad.
- 9 Explicar con SHAP.
- 10 Congelar modelo y reproducibilidad.
- 11 Exponer el modelo con FastAPI.
- 12 Construir portal web como fase posterior.

18. Bibliografía y fuentes de valor

UCI Online Retail II, Chen (2012), DOI 10.24432/C5CG6D. ■cite■turn0search1■

Ibrahim et al. (2026), Scientific Reports: RFM + clustering + deep learning para churn e-commerce. ■cite■turn0search0■

Zaghloul, Barakat & Rezk (2025), Expert Systems with Applications: RFM, K-means, LSTM/GRU y modelos tradicionales. ■cite■turn0search5■

Annals of Operations Research (2023/2024): RFM variable en el tiempo, RNN y transformers. ■cite■turn0search6■

Estudio 2026 sobre UCI Online Retail II: separación temporal 18/6 meses para churn. ■cite■turn0search2■

Frontiers in Artificial Intelligence (2026): churn explicable, SHAP, calibración y métricas de negocio. ■cite■turn0search14■

Scientific Reports (2025): desbalance, ensembles y SMOTE. ■cite■turn0search16■

Procedia Computer Science (2026): MLOps end-to-end para churn. ■cite■turn0search18■

Decisión metodológica central: todavía no entrenamos el modelo definitivo. Primero auditamos los datos reales y congelamos la definición temporal de churn. Así evitamos construir un modelo técnicamente sofisticado sobre una etiqueta mal definida o con leakage.